PROGRAMA DE FORMAÇÃO ACELERADA - *Bolsa Futuro Digital*
Curso de Desenvolvimento Front-end

PROJETO – CONSULTA METEREOLÓGICA

1. Contextualização

Consultar a previsão do tempo pode ajudar no planejamento de atividades cotidianas, como decidir o que vestir, organizar uma viagem ou escolher o melhor momento para realizar uma atividade ao ar livre. Neste projeto, você deverá desenvolver uma aplicação web de **consulta meteorológica**, que permitirá consultar informações climáticas de diferentes cidades e acompanhar a previsão do tempo para os próximos dias.

Diferentemente de aplicações que trabalham apenas com dados cadastrados pelo próprio usuário, o aplicativo deverá obter suas informações de uma API pública. Isso significa que a aplicação precisará realizar requisições pela internet, aguardar as respostas do serviço e apresentar os dados recebidos na interface.

Além da consulta meteorológica, a aplicação deverá permitir que o usuário mantenha uma lista de cidades favoritas. Essa lista deverá ser armazenada no próprio navegador, utilizando o localStorage.

2. Objetivos

O objetivo principal é desenvolver uma aplicação fron-tend que integre comunicação assíncrona com uma API pública, manipulação do DOM e armazenamento local.

Ao finalizar o projeto, espera-se que você consiga:

- realizar requisições HTTP utilizando fetch();
- compreender e utilizar operações assíncronas com Promises e async/await;
- interpretar respostas de uma API no formato JSON;
- utilizar dados recebidos de um serviço externo para construir elementos HTML dinamicamente;
- trabalhar com duas requisições dependentes uma da outra;
- tratar situações de carregamento, sucesso e erro;
- validar dados fornecidos pelo usuário;
- utilizar arrays e funções para organizar a lógica da aplicação;
- armazenar e recuperar informações utilizando localStorage;
- desenvolver uma interface que forneça feedback adequado ao usuário durante as operações assíncronas.

3. API meteorológica

Para obter as informações, será utilizada a API pública **Open-Meteo**.

Documentação oficial:

- Geocodificação: https://open-meteo.com/en/docs/geocoding-api
- Previsão meteorológica: https://open-meteo.com/en/docs

A Open-Meteo disponibiliza serviços para localizar cidades e consultar dados meteorológicos. Seu acesso gratuito para uso não comercial dispensa chave de API, e o serviço oferece suporte a CORS (*Cross-Origin Resource Sharing* ou Compartilhamento de Recursos entre Origens), permitindo sua utilização diretamente no navegador. Os dados apresentados deverão incluir a atribuição à fonte, conforme as condições de uso da API.

3.1 As duas etapas de consulta

O funcionamento principal da aplicação deverá envolver duas etapas assíncronas.

Primeira etapa: localizar a cidade.

O usuário informa o nome de uma cidade. A aplicação consulta o serviço de geocodificação da Open-Meteo, que retorna os locais correspondentes e informações como nome, país, região e coordenadas geográficas.

Segunda etapa: consultar o clima.

Após selecionar ou identificar a cidade desejada, a aplicação utiliza suas coordenadas para consultar as condições meteorológicas e a previsão para os próximos dias.

A segunda requisição depende das informações obtidas na primeira. Portanto, a aplicação deverá garantir que as coordenadas estejam disponíveis antes de solicitar a previsão.

As consultas deverão ser realizadas por meio de requisições HTTP, utilizando fetch() e tratamento apropriado das respostas.

4. Requisitos funcionais

RF01 — Pesquisa de cidades

A aplicação deverá possuir um formulário que permita ao usuário informar o nome de uma cidade. Após o envio, o sistema deverá consultar a API de geocodificação e apresentar os locais encontrados.

Cada resultado deverá exibir, quando disponíveis:

- nome da cidade;
- estado;
- país;
- informação suficiente para distinguir localidades com nomes semelhantes.

Se houver mais de um resultado, o usuário deverá conseguir identificar e selecionar a cidade desejada.

A aplicação não deverá assumir que o primeiro resultado corresponde necessariamente à localidade pretendida.

RF02 — Seleção da cidade

Quando houver resultados de pesquisa, o usuário deverá poder selecionar uma cidade para consultar sua previsão.

A aplicação deverá utilizar as coordenadas geográficas retornadas pela API, e não tentar deduzir latitude e longitude a partir do nome da cidade. Após a seleção, deverá ser iniciada a consulta meteorológica correspondente.

RF03 — Exibição das condições atuais

Após uma consulta bem-sucedida, a aplicação deverá apresentar um painel com as principais informações meteorológicas disponíveis.

O painel deverá conter, no mínimo:

- nome da cidade e país;
- temperatura atual;
- descrição da condição meteorológica;
- velocidade do vento;
- sensação térmica, quando disponível;
- umidade relativa do ar, quando disponível.

Os dados deverão ser apresentados em unidades compreensíveis, com identificação adequada das grandezas. A descrição das condições meteorológicas deverá ser construída a partir dos códigos retornados pela API. Para isso, você deverá implementar uma associação entre os códigos meteorológicos e descrições legíveis, conforme a documentação oficial. Por exemplo, um código poderá ser associado a uma descrição como “Céu limpo”, “Parcialmente nublado” ou “Chuva”.

RF04 — Previsão para os próximos dias

Além das condições atuais, a aplicação deverá apresentar a previsão meteorológica para os próximos cinco dias, incluindo o dia atual quando ele fizer parte do período retornado.

Para cada dia, deverão ser exibidas:

- data;
- descrição da condição meteorológica;
- temperatura mínima;
- temperatura máxima;
- probabilidade máxima de precipitação, quando disponível.

A previsão deverá ser apresentada em cartões ou em outra organização visual que permita comparar os dias. As informações deverão ser obtidas da resposta da API, sem a criação de valores fictícios para preencher campos ausentes.

RF05 — Atualização da consulta

O usuário deverá poder pesquisar outra cidade a qualquer momento. Quando uma nova cidade for selecionada, a aplicação deverá atualizar as informações meteorológicas exibidas. Os dados da consulta anterior não poderão ser apresentados como se pertencessem à nova cidade. Caso a nova consulta falhe, a aplicação deverá informar o problema e evitar que o usuário confunda os dados anteriores com os resultados da pesquisa atual.

RF06 — Cidades favoritas

A aplicação deverá permitir que o usuário adicione uma cidade à sua lista de favoritas. A inclusão deverá ocorrer por meio de um botão ou controle equivalente, associado à cidade selecionada. A lista de favoritas deverá apresentar, no mínimo, o nome da cidade e o país, além de uma opção para removê-la. O usuário deverá poder selecionar uma cidade favorita para consultar novamente suas condições meteorológicas. Não deverá ser possível cadastrar repetidamente a mesma cidade como favorita. Para identificar duplicidades, deverão ser considerados dados suficientes para distinguir localidades diferentes, como as coordenadas geográficas e a identificação da localidade.

RF07 — Persistência dos favoritos

A lista de cidades favoritas deverá ser armazenada no localStorage.

Ao abrir ou atualizar a aplicação, os favoritos previamente salvos deverão ser recuperados e apresentados automaticamente.

As seguintes operações deverão atualizar o armazenamento:

- adicionar uma cidade favorita;
- remover uma cidade favorita.

A aplicação deverá funcionar mesmo quando não houver nenhuma cidade favorita cadastrada. Se os dados armazenados estiverem ausentes ou inválidos, o sistema deverá tratar a situação sem interromper a execução.

RF08 — Feedback durante as operações

A aplicação deverá fornecer informações visuais sobre o estado das operações assíncronas.

Deverão ser contempladas, no mínimo, as seguintes situações:

- aguardando uma pesquisa;
- carregando as condições meteorológicas;
- consulta concluída;
- nenhum resultado encontrado;
- erro ao localizar a cidade;
- erro ao consultar a previsão.

As mensagens deverão ser apresentadas em uma área apropriada da interface. A indicação de carregamento deverá desaparecer quando a operação terminar, independentemente de ter sido concluída com sucesso ou com erro.

5. Requisitos de comunicação assíncrona

Esta seção representa uma parte central da avaliação.

RA01 — Utilização de Fetch API

As consultas deverão ser realizadas com fetch().

Não será permitido substituir as consultas reais por dados previamente escritos no código, nem utilizar respostas simuladas como implementação definitiva.

RA02 — Tratamento das Promises

O código deverá tratar adequadamente as operações assíncronas.

Poderá ser utilizada a sintaxe async/await ou a composição de Promises com .then() e .catch(), conforme os conhecimentos trabalhados em aula. O código deverá aguardar a conclusão das requisições antes de tentar utilizar seus resultados.

RA03 — Processamento de JSON

A resposta da API deverá ser convertida para JSON e seus dados deverão ser interpretados conforme a estrutura documentada pelo serviço.

A aplicação deverá acessar os campos necessários para localizar cidades e apresentar as condições meteorológicas.

RA04 — Tratamento de erros HTTP e de comunicação

O código deverá distinguir uma resposta HTTP malsucedida de uma resposta bem-sucedida.

É importante observar que fetch() não rejeita automaticamente sua Promise apenas porque o servidor respondeu com um código HTTP de erro, como 404. Portanto, a aplicação deverá verificar a propriedade ok da resposta e tratar os erros adequadamente.

Também deverão ser tratados problemas como:

- falhas de conexão;
- indisponibilidade do serviço;
- respostas inválidas ou inesperadas;
- ausência de resultados para a pesquisa;
- falhas durante a conversão ou interpretação dos dados, quando pertinente.

RA05 — Dependência entre requisições

A consulta meteorológica deverá ocorrer somente após a identificação da cidade e a obtenção de suas coordenadas. A implementação deverá respeitar a ordem lógica das operações, evitando tentar consultar o clima antes de concluir a geocodificação.

RA06 — Atualização do DOM

Os resultados deverão ser apresentados dinamicamente utilizando JavaScript. Não será suficiente escrever previamente no HTML os dados que deveriam vir da API. A aplicação deverá criar, preencher, atualizar ou remover elementos conforme as informações recebidas e as ações realizadas pelo usuário.

6. Requisitos de interface

A interface deverá ser clara, organizada e fácil de utilizar. A página deverá apresentar, no mínimo, as seguintes áreas:

Cabeçalho com o nome da aplicação.

1. Formulário de pesquisa de cidades.
2. Área de resultados da pesquisa de localidades.
3. Painel das condições meteorológicas atuais.
4. Área de previsão para os próximos dias.
5. Área de cidades favoritas.
6. Área para mensagens de carregamento, sucesso e erro.

A organização visual deverá utilizar CSS para definir cores, tipografia, espaçamentos e disposição dos elementos. Você poderá escolher livremente o estilo visual, desde que os requisitos funcionais sejam atendidos e a interface mantenha uma hierarquia visual consistente. A aplicação deverá apresentar mensagens apropriadas quando ainda não houver cidade selecionada, quando não existirem favoritos ou quando uma pesquisa não retornar resultados.

7. Requisitos técnicos e organização do projeto

O projeto deverá ser organizado, preferencialmente, com a seguinte estrutura:

```
sistema/
├── index.html
├── css/
│   └── style.css
└── js/
    └── script.js
```

O código JavaScript deverá ser organizado em funções que representem responsabilidades distintas.

Entre as responsabilidades esperadas, estão:

- pesquisar cidades;
- apresentar resultados de geocodificação;
- consultar as condições meteorológicas;
- apresentar as condições atuais;
- apresentar a previsão diária;
- converter códigos meteorológicos em descrições;
- carregar os favoritos;
- salvar os favoritos;
- adicionar e remover favoritos;
- atualizar mensagens de estado.

Esses nomes são apenas sugestões. Você poderá escolher outros nomes e uma organização diferente, desde que o código permaneça compreensível e suficientemente modular. Não será exigido que todas essas responsabilidades sejam implementadas em funções separadas, mas a solução **deverá evitar** concentrar toda a lógica em um único bloco de código.

8. Validações e regras de negócio

A aplicação deverá obedecer às seguintes regras:

1. A pesquisa não deverá ser executada se o campo da cidade estiver vazio ou contiver apenas espaços.
2. A seleção de uma localidade deverá preceder a consulta meteorológica.
3. Os dados apresentados deverão corresponder à cidade selecionada.
4. Uma pesquisa sem resultados deverá produzir uma mensagem apropriada.
5. A lista de favoritos não poderá conter duplicatas da mesma localidade.
6. A remoção de um favorito deverá atualizar a interface e o localStorage.
7. A lista de favoritos deverá continuar disponível após atualizar a página.
8. A ausência de favoritos não poderá provocar erros.

9. A aplicação não deverá apresentar dados meteorológicos fictícios quando a API não fornecer uma informação.
10. Uma falha de comunicação deverá ser informada ao usuário sem deixar a interface permanentemente em estado de carregamento.

9. Cenários de teste

Antes de entregar o projeto, você deverá executar os seguintes testes e verificar os resultados.

Teste 1 — Pesquisa válida

Pesquise uma cidade conhecida, como Porto Alegre. Verifique se a aplicação apresenta os resultados de localização e permite selecionar a cidade desejada.

Teste 2 — Localidades com nomes semelhantes

Pesquise um nome de cidade que corresponda a mais de uma localidade. Verifique se os resultados permitem distinguir as localidades e selecionar a correta.

Teste 3 — Pesquisa vazia

Tente pesquisar sem preencher o campo ou informando somente espaços. A aplicação deverá impedir a requisição e apresentar uma orientação adequada.

Teste 4 — Cidade não encontrada

Pesquise um nome que não corresponda a nenhuma localidade encontrada pela API. A aplicação deverá informar que não foram encontrados resultados, sem exibir dados meteorológicos incorretos.

Teste 5 — Consulta meteorológica

Selecione uma cidade válida. Verifique se as condições atuais e a previsão dos próximos dias são exibidas com as respectivas datas e unidades.

Teste 6 — Nova pesquisa

Após consultar uma cidade, pesquise outra. Verifique se os dados anteriores são substituídos pelos novos resultados e se a identificação da cidade corresponde à consulta atual.

Teste 7 — Carregamento

Observe a interface durante uma consulta. Verifique se existe uma indicação visual de carregamento e se ela desaparece ao término da operação.

Teste 8 — Falha de comunicação

Simule uma indisponibilidade da rede, por exemplo, utilizando as ferramentas de desenvolvimento do navegador para bloquear a conexão. Verifique se a aplicação apresenta uma mensagem de erro e permite realizar uma nova tentativa.

Teste 9 — Adicionar favorito

Consulte uma cidade e adicione-a aos favoritos. Verifique se ela aparece na lista e se os dados foram armazenados no localStorage.

Teste 10 — Persistência

Adicione algumas cidades favoritas e atualize a página. Verifique se os favoritos continuam disponíveis.

Teste 11 — Remover favorito

Remova uma cidade da lista. Verifique se ela desaparece da interface e se também deixa de constar no armazenamento local.

Teste 12 — Reutilizar um favorito

Selecione uma cidade já salva nos favoritos. Verifique se a aplicação utiliza os dados de localização armazenados para realizar uma nova consulta meteorológica.

Teste 13 — Dados ausentes

Verifique como a aplicação se comporta quando um campo opcional não está disponível na resposta recebida. O sistema deverá continuar funcionando e apresentar uma indicação apropriada para a informação ausente.

12. Desafios opcionais

Depois de concluir os requisitos obrigatórios, você poderá ampliar a aplicação com funcionalidades adicionais, como:

- pesquisa de cidades favoritas por nome;
- remoção de todos os favoritos mediante confirmação;
- apresentação de uma previsão horária;
- exibição de indicadores adicionais, como precipitação e índice UV;
- alternância entre unidades de temperatura;
- comparação das condições meteorológicas entre duas cidades;
- histórico das últimas cidades consultadas;
- atualização manual da previsão;
- prevenção de consultas duplicadas enquanto uma requisição equivalente estiver em andamento.

Esses recursos são opcionais e não substituem os requisitos obrigatórios.